Established – 1961 Subject:_____

SEVA SADAN'S

R. K. TALREJA COLLEGE

OF

ARTS, SCIENCE & COMMERCE ULHASNAGAR – 421003

CERTIFICATE

This is to certify that Mr./Ms. _____ of S.Y. Information Technology (SYIT) Roll No. _____ has satisfactorily completed the Open Source DataBase Management System Mini Project entitled _____ during the academic year 2025 – 2026, as a part of the practical requirement. The project work is found to be satisfactory and is approved for submission.

PROF. INCHARGE **HEAD OF DEPT**

_____ _____

INDEX

Sr. No.	Chapter Title	PAGE NO
1	Introduction	
2	Problem Definition	
3	Objectives of the Project	
4	Scope of the Project	
5	Requirement Specification	
6	System Design	
7	Database Design	
8	UML Diagrams	
9	SQL Implementation	
10	System Testing and Result	
11	Security, Backup and Recovery	
12	Future Scope and Conclusion	
13	References	
14	Glossary	

1.INTRODUCTION

Market research plays a crucial role in understanding customer preferences, product performance, and market trends. Organizations collect large amounts of data from customers through surveys and feedback forms. Managing such data manually is inefficient, time-consuming, and prone to errors. Therefore, a computerized database system is required to store, manage, and analyze this information effectively.
This project presents a Market Research Data Management Database developed using MySQL. The system stores customer details, product information, survey records, and feedback data in structured relational tables. MySQL, an open-source relational database management system, is used because of its reliability, performance, and wide industry adoption.

The database allows users to insert new records, update existing information, delete outdated data, and retrieve reports using SQL queries. It ensures accuracy by applying integrity constraints such as primary keys, foreign keys, and unique constraints. Transactions guarantee that related operations (such as survey submission and feedback entry) are completed fully or not at all.-**-

Additionally, the system supports multi-user access through concurrency control mechanisms, preventing conflicts when multiple users submit feedback simultaneously. The database is normalized up to Third Normal Form (3NF) to eliminate redundancy and maintain consistency.

Overall, this project demonstrates the practical application of database management concepts in solving real-world problems related to market research data handling.

-

2. PROBLEM DEFINITION

In many small organizations and academic projects, market research data is maintained manually using paper records or spreadsheets. This traditional approach creates several serious problems that affect data quality and usability.
Manual systems often lead to data duplication because the same customer information may be recorded multiple times. Inconsistency occurs when different versions of the same data exist in different places. Searching for specific records becomes slow and inefficient, especially as the volume of data increases. Updating information requires editing multiple files, increasing the chance of errors.

Security is another major concern. Physical records can be lost, damaged, or accessed by unauthorized persons. There is also a high risk of data loss due to accidents such as fire, theft, or hardware failure when proper backup systems are not in place.

Furthermore, generating reports from manual data is difficult and time-consuming. Organizations cannot quickly analyze customer feedback or product performance, which limits decision-making capabilities.

To overcome these issues, a structured database system is required. The proposed solution is a relational database using MySQL that ensures efficient storage, fast retrieval, data integrity, security, and support for multiple users.

-

3. OBJECTIVES OF THE PROJECT

The main objective of this project is to design and implement a relational database system that efficiently manages market research data in an organized and automated manner. The system replaces manual data handling with a structured solution that improves accuracy, reduces redundancy, and enables quick access to information. By using a MySQL database, the project demonstrates how real-world survey and feedback data can be stored, processed, and analyzed securely and reliably.

*** To design a structured database schema with related tables such as Customer, Product, Survey, and Feedback.**

*** To reduce data redundancy and improve consistency through normalization (up to Third Normal Form — 3NF).**

*** To enable efficient data retrieval, insertion, updating, and deletion using SQL queries.**

*** To maintain data integrity using constraints like Primary Key, Foreign Key, Unique, and Not Null.**

*** To support safe transactions for operations such as survey submission and feedback entry.**

*** To allow multiple users to access the database simultaneously without conflicts (concurrency support).**

*** To ensure security, reliability, and backup mechanisms for protecting valuable research data.**

-

4. SCOPE OF THE PROJECT

This project focuses on designing and implementing a relational database system to manage market research data in a structured and efficient manner. The system is intended for academic purposes and small-scale organizations where research data needs to be stored, organized, and analyzed without relying on complex enterprise tools. It replaces manual record-keeping methods with a computerized solution that improves accuracy, consistency, and accessibility of information. The database manages key entities such as customers, products, surveys, and feedback, enabling users to conduct research activities systematically. By applying normalization, constraints, and relational dependencies, the system ensures minimal redundancy and reliable data storage.

The scope defines what the system will cover and what functionalities it will provide within the project boundaries.

*** The system stores detailed information about customers participating in market research, including demographic details and contact information.**
*** It maintains a product database containing items or services that are being evaluated through surveys.**
*** The project supports recording of survey sessions conducted with customers for specific products.**
*** It captures feedback provided by customers, including ratings, reviews, and comments.**
*** The database allows CRUD operations (Create, Read, Update, Delete) for efficient data management.**
*** It enforces data integrity using primary keys, foreign keys, unique constraints, and validation rules.**
*** The system uses normalization (up to Third Normal Form) to reduce redundancy and maintain consistency.**
*** SQL queries enable fast retrieval of information for analysis and report generation.**
*** The project is limited to backend database operations and does not include a full web or mobile interface.**

-

*** It serves as a foundational system that can be expanded into a larger enterprise solution in the future.**

Applications of the Project

This database system can be applied in various real-world scenarios where structured feedback collection and analysis are required.

*** Market research agencies can use the system to store and analyze customer opinions about products or services.**
*** Small businesses can evaluate customer satisfaction and product performance using collected feedback.**
*** Educational institutions can use the project as a practical example to teach database design and SQL implementation.**
*** Startups can conduct product validation studies before launching new offerings in the market.**
*** Retail companies can analyze consumer preferences to improve product development and marketing strategies.**
*** Survey organizations can manage participant information and responses efficiently.**
*** Government or non-profit organizations can collect public opinion data for policy or social research.**
*** Companies can use the system to track product improvements based on customer feedback trends.**
*** Data analysts can extract insights from stored data using SQL queries and reports.**
*** The system can serve as a backend for future web-based or mobile survey applications.**
*** Training programs in database management can use this project as a demonstration model.**
*** Research students can extend the system for advanced analytics, dashboards, or decision-support systems.**

-

-

Limitations of the Project

Despite its usefulness, the project has certain limitations due to its academic and small-scale nature.

*** The system does not include an online interface for real-time data collection through web or mobile applications.**
*** Advanced analytics such as predictive modeling, machine learning, or AI-based insights are not implemented.**
*** Security features are basic and do not include advanced encryption or multi-factor authentication.**
*** The project does not handle very large datasets typical of enterprise-level systems.**
*** Real-time processing and streaming data capabilities are not supported.**
*** Reporting features are limited to SQL query outputs rather than interactive dashboards.**
*** The system assumes controlled access and does not fully address distributed database challenges.**
*** Backup and recovery mechanisms are basic and not automated.**
*** Integration with external systems such as CRM, ERP, or cloud platforms is not included.**
*** User roles and permission management are minimal compared to commercial software solutions.**
*** Performance optimization techniques for high-traffic environments are not extensively implemented.**
*** The project focuses mainly on database design rather than full application development.**

-

5. REQUIREMENT SPECIFICATION

Hardware Requirements

- Processor: Intel i3 or higher
- RAM: Minimum 4 GB
- Storage: At least 10 GB free disk space

Software Requirements

- MySQL Server
- MySQL Workbench
- Operating System: Windows or Linux

Functional Requirements

The system must allow users to:

- Add, update, and delete customer information
- Store product details
- Record surveys conducted
- Capture feedback for products
- Retrieve reports using queries

Non-Functional Requirements

- High data accuracy
- Security against unauthorized access
- Reliability and consistency
- Fast response time

-

6. SYSTEM DESIGN

he system is designed to manage market research data efficiently using a relational database approach. It replaces manual record keeping with an automated solution that ensures accuracy, consistency, and easy retrieval of information related to customers, products, surveys, and feedback. The database is implemented using **MySQL, and users interact with it through **MySQL Workbench**, which provides tools for executing SQL queries and managing database objects. The design emphasizes simplicity, reliability, and data integrity, making it suitable for academic and small-scale applications.**

Architecture Overview

The system follows a two-tier architecture:

Client Layer : Users interact through SQL queries using MySQL Workbench.
Database Layer : MySQL server stores data, processes queries, and enforces constraints.

Modular Design

The system consists of four main modules:

Customer Management : Stores participant details (name, age, gender, email).
Product Management : Maintains information about products being evaluated.
Survey Management : Records survey sessions conducted with customers.

-

Feedback Management : Captures ratings and comments for products.

Data Flow

*** Customer and product data are stored first.**
*** Surveys link customers to research sessions.**
*** Feedback connects surveys with products.**
*** Reports are generated using SQL joins across tables.**

Transaction Design

Survey submission and feedback entry are executed as a single transaction to maintain atomicity.

Concurrency Control

*** Supports multiple users simultaneously.**
*** Uses locking and isolation levels to prevent conflicts.**
*** Ensures consistent results even during concurrent updates.**

Integrity Constraints

*** Primary keys uniquely identify records.**
*** Foreign keys maintain relationships.**
*** NOT NULL and UNIQUE constraints prevent invalid or duplicate data.**
*** Referential integrity avoids orphan records.**

Normalization

The database is normalized up to Third Normal Form (3NF) to reduce redundancy, eliminate anomalies, and ensure efficient storage.

-

Reliability and Performance

*** Supports fast query processing.**

*** Indexing can improve search performance.**

*** Backup mechanisms ensure data safety.**

-

7. DATABASE DESIGN

7.1 Entities

The database system consists of the following main entities:

Customer
Represents individuals who participate in market research surveys. It stores personal and contact details such as name, age, gender, and email.

Product
Represents the products or services being evaluated during the research process. It stores product name and category information.

Survey
Represents survey sessions conducted with customers. Each survey links a specific customer to a particular research activity and records the survey date.

Feedback
Stores customer responses for products, including ratings and comments given during a survey.

7.2 Table Structure

Customer Table
This table stores personal details of survey participants.

Attribute	Data Type	Description
Customer_id	INT	**Primary key, uniquely identifies each**

-

Customer Name	VARCHAR(100)	**Name of the customer**
Age	INT	**Age of the customer**
Gender	VARCHAR(100)	**Gender of the customer**
Email	VARCHAR(100)	**Unique email address**

Product Table

This table stores information about products being evaluated.

Attribute	Data type	Description
product_id	INT	**Primary key, uniquely identifies each product**
product_name	**VARCHAR(100)**	**Name of the product**
category	**VARCHAR(50)**	**Product category**

Survey Table

This table records survey sessions conducted with customers.

Attribute	Data Type	Description
survey_id	INT	**Primary key, uniquely identifies each survey**
survey_date	DATE	**Date of the survey**
customer_id	INT	**Foreign key referencing Customer table**

-

Feedback Table

This table stores customer feedback on products.

Attribute	Data Type	Description
feedback_id	INT	**Primary key, uniquely identifies each feedback**
Survey_id	INT	**Foreign key referencing Survey table**
product_id	INT	**Foreign key referencing Product table**
rating	INT	**Rating given to the product**
comments	TEXT	**Customer comments**

Normalization

The schema is normalized up to *Third Normal Form (3NF)* to reduce redundancy and improve data consistency.

First Normal Form (1NF) :
All attributes are atomic, and there are no repeating groups.

Second Normal Form (2NF) :
No partial dependencies exist; non-key attributes fully depend on the primary key.

Third Normal Form (3NF) :No transitive dependencies; non-key

-

attributes depend only on the primary key.

Integrity Constraints

1. Primary Key Constraint

* Uniquely identifies each record in a table.
* Prevents duplicate and NULL values.
* Examples:

  * customer_id in Customer table
  * product_id in Product table
  * survey_id in Survey table
* feedback_id in Feedback table

2. Foreign Key Constraint

* Establishes relationships between tables.
* Ensures referential integrity (related parent record must exist).
* Examples:

  * customer_id in Survey table references Customer table
  * survey_id in Feedback table references Survey table
  * product_id in Feedback table references Product table

3. NOT NULL Constraint

* Ensures mandatory fields are not left empty.
* Examples:

  * Customer name and email
  * Product name
  * Survey date

-

*** Feedback rating**

4. UNIQUE Constraint

*** Prevents duplicate values in specific columns.**
*** Maintains data accuracy.**
*** Example:**

*** Email field in Customer table**

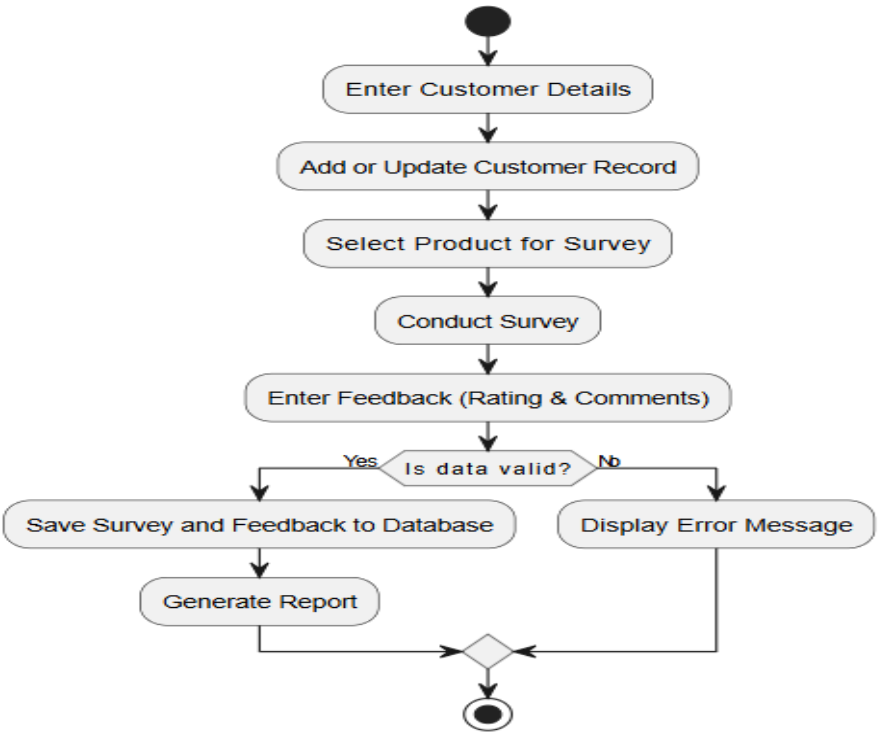
5. Data Type Constraints

*** Appropriate data types such as INT, VARCHAR, DATE, and TEXT are used.**
*** Ensures correct storage format and better performance.**

8. UML DIAGRAMS

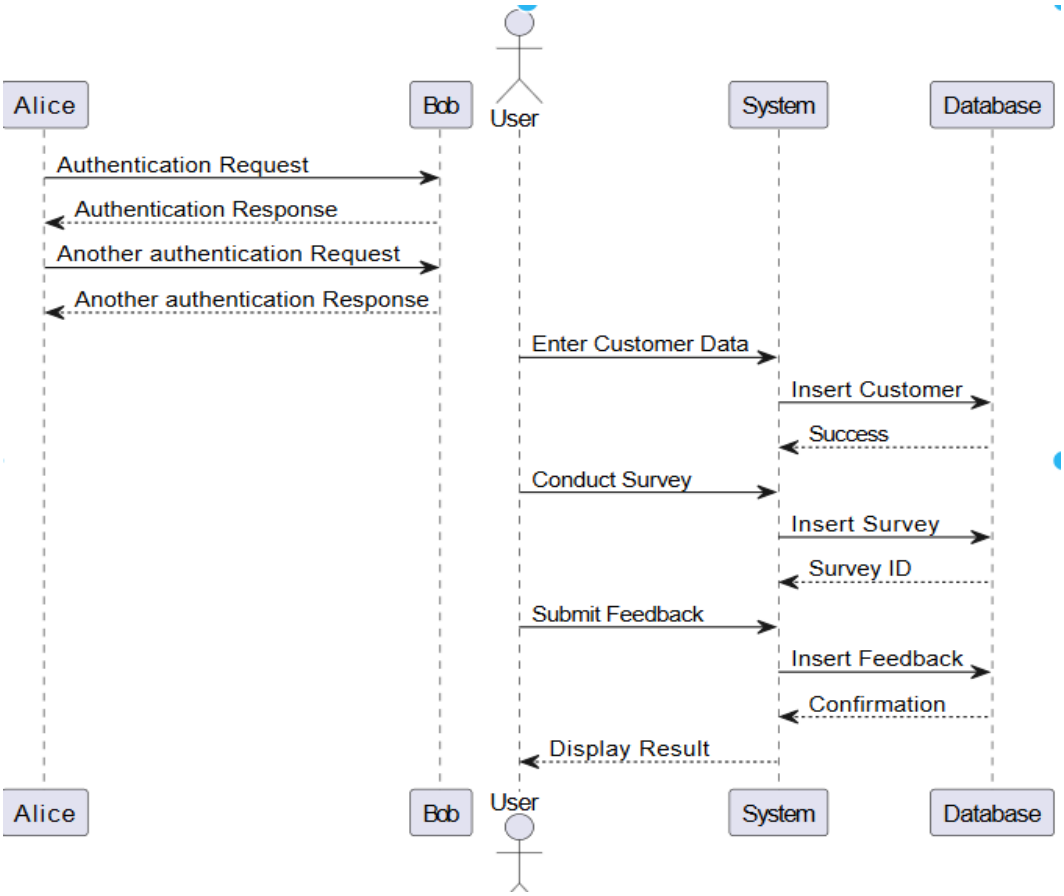
-

8.1 ER Diagram

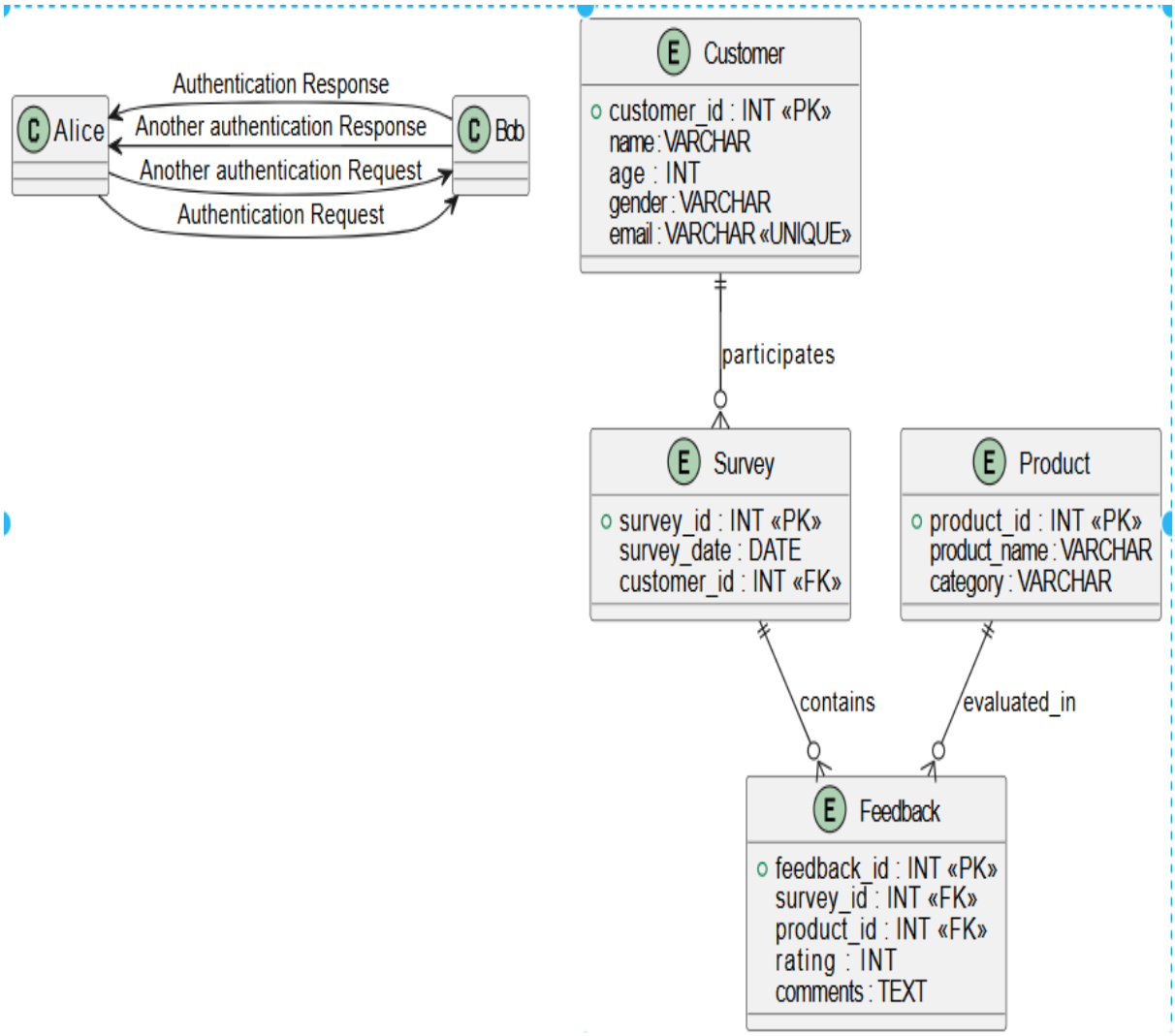


8.1 ER Diagram

8.2 Use Case Diagram

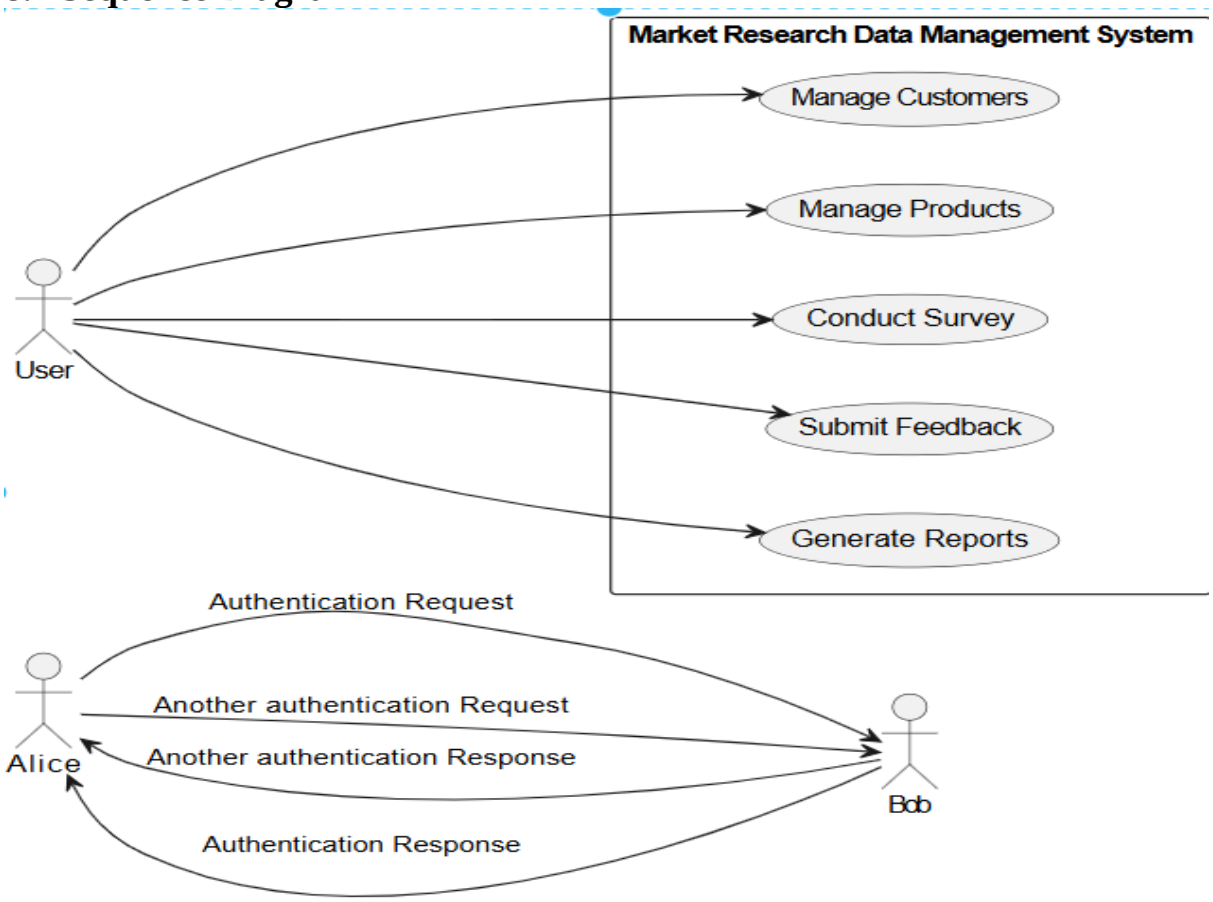


8.3Activity Diagram



-

8.4 Sequence Diagram



Entities:

- Customer
- Survey
- Feedback
- Product

Relationships:

- One customer can have many surveys
- One survey can have feedback
- Each feedback relates to one product

-

9.SQL Implementation

9.1 Create Database

sql

```
CREATE DATABASE MarketResearchDB;
USE MarketResearchDB;
```

9.2 Table Creation

Customer Table

sql

```
CREATE TABLE Customer (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT,
    gender VARCHAR(10),
    email VARCHAR(100) UNIQUE NOT NULL
);
```

Product Table

sql

```
CREATE TABLE Product (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category VARCHAR(50)
);
```

Survey Table

sql

```
CREATE TABLE Survey (
    survey_id INT AUTO_INCREMENT PRIMARY KEY,
    survey_date DATE NOT NULL,
    customer_id INT NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES
```


-

```
Customer(customer_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Feedback Table

sql

```
CREATE TABLE Feedback (
    feedback_id INT AUTO_INCREMENT PRIMARY KEY,
    survey_id INT NOT NULL,
    product_id INT NOT NULL,
    rating INT CHECK (rating BETWEEN 1 AND 5),
    comments TEXT,
    FOREIGN KEY (survey_id) REFERENCES
Survey(survey_id)
        ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES
Product(product_id)
);
```

9.3 Data Insertion

Insert Customers

sql

```
INSERT INTO Customer (name, age, gender, email) VALUES
('Rahul Sharma', 25, 'Male', 'rahul@example.com'),
('Sahil Verma', 22, 'Male', 'sahil@example.com'),
('Aditi Mehta', 24, 'Female', 'aditi@example.com'),
('Neha Singh', 23, 'Female', 'neha@example.com'),
('Rohit Jain', 26, 'Male', 'rohit@example.com');
```

Insert Products

sql

```
INSERT INTO Product (product_name, category) VALUES
('Smartphone X', 'Electronics'),
('Organic Shampoo', 'Personal Care'),
```


-

('Sports Shoes', 'Footwear');

Insert Survey Records

sql
```
INSERT INTO Survey (survey_date, customer_id) VALUES
('2025-02-10', 1),
('2025-02-12', 2),
('2025-02-15', 3);
```

Insert Feedback
sql
```
INSERT INTO Feedback (survey_id, product_id, rating,
comments) VALUES
(1, 1, 5, 'Excellent product'),
(2, 2, 4, 'Good quality'),
(3, 3, 3, 'Average experience');
```

9.4 SQL Queries and Advanced Concepts

Join Query

sql
```
SELECT c.name, p.product_name, f.rating
FROM Customer c
JOIN Survey s ON c.customer_id = s.customer_id
JOIN Feedback f ON s.survey_id = f.survey_id
JOIN Product p ON f.product_id = p.product_id;
```

Group By and Having Clause
sql
```
SELECT product_id, AVG(rating) AS average_rating
FROM Feedback
GROUP BY product_id
HAVING average_rating > 3;
```


-

9.5 View Creation

sql

```sql
CREATE VIEW Product_Summary AS
SELECT
   p.product_name,
   AVG(f.rating) AS average_rating
FROM Product p
JOIN Feedback f ON p.product_id = f.product_id
GROUP BY p.product_name;
SELECT * FROM Product_Summary;
```

9.6 Transaction Handling

Successful Transaction (Survey + Feedback)

sql

```sql
START TRANSACTION;
INSERT INTO Survey (survey_date, customer_id)
VALUES ('2025-02-20', 4);
INSERT INTO Feedback (survey_id, product_id, rating,
comments)
VALUES (LAST_INSERT_ID(), 1, 5, 'Very satisfied');
COMMIT;
```

Rollback Transaction

sql

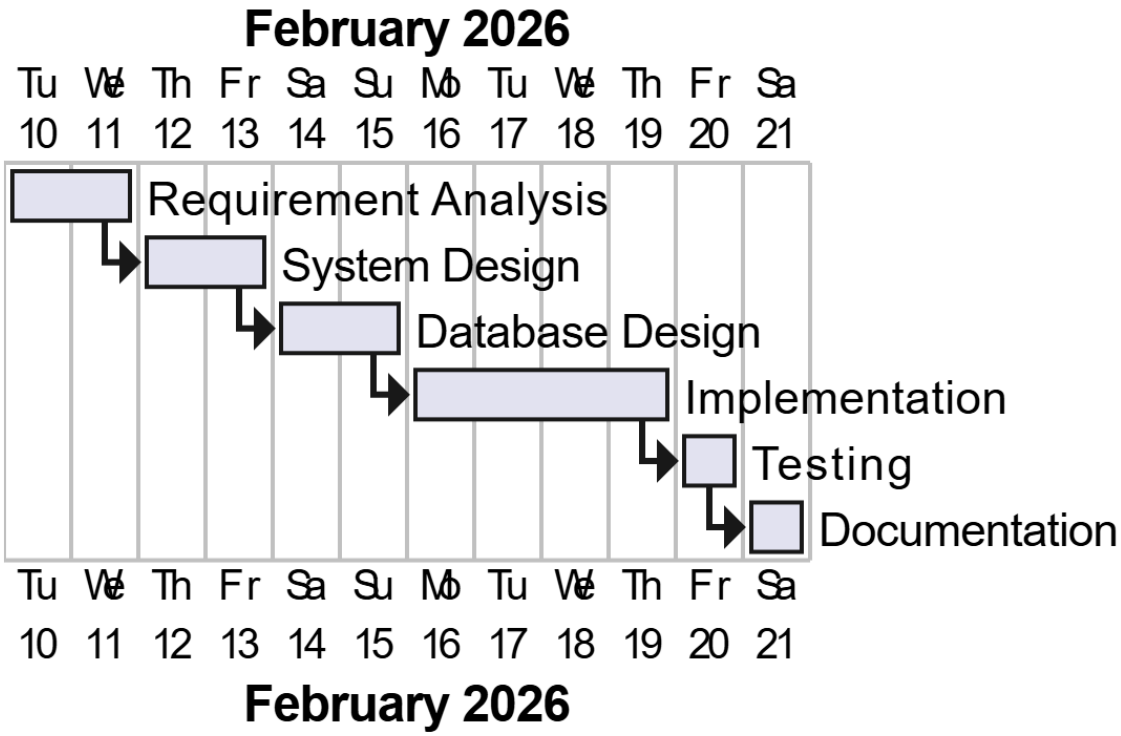
```sql
START TRANSACTION;

INSERT INTO Feedback (survey_id, product_id, rating,
comments)
VALUES (1, 2, 10, 'Invalid rating test');

ROLLBACK;
```


-

GANTT CHAT


February 2026
Tu We Th Fr Sa Su Mo Tu We Th Fr Sa
10 11 12 13 14 15 16 17 18 19 20 21
Requirement Analysis
System Design
Database Design
Implementation
Testing
Documentation
Tu We Th Fr Sa Su Mo Tu We Th Fr Sa
10 11 12 13 14 15 16 17 18 19 20 21
February 2026

-

10. SYSTEM TESTING AND RESULT

System testing was conducted to verify the correctness, reliability, and performance of the Market Research Database Management System. Each database component was tested using various SQL queries to ensure that the system functions as expected and meets the project objectives.

• Data Definition Language (DDL) queries such as CREATE, ALTER, and DROP were executed to verify proper creation, modification, and deletion of database objects.
• Data Manipulation Language (DML) queries including INSERT, UPDATE, and DELETE were tested to ensure correct data entry, modification, and removal.
• Data Query Language (DQL) statements, mainly SELECT queries, were used to retrieve data and confirm that records were fetched accurately based on specified conditions.
• Queries were tested with valid inputs to ensure successful execution and correct output.
• Invalid inputs were also used to verify error detection, constraint enforcement, and system stability.
• Complex queries involving conditions, sorting, and filtering were executed to check logical correctness.
• Performance of frequently used queries was observed to ensure efficient data retrieval.

10.1 Query Testing

1. Database and Table Creation Testing

Database and Table Creation Testing is performed to verify that the database and all required tables are created successfully without errors. This testing ensures that the database structure is correctly defined before inserting or retrieving any data.

Actual Result

The database MarketResearchDB (Customer, Product, Survey, Feedback) were created successfully.

-

Customer Table

```
mysql> select*from Customer;
+------------+--------------+------+--------+------------------+
| customer_id | name         | age  | gender | email            |
+------------+--------------+------+--------+------------------+
|          1 | Rahul Sharma |   25 | Male   | rahul@example.com |
|          2 | Sahil Verma  |   22 | Male   | sahil@example.com |
|          3 | Aditi Mehta  |   24 | Female | aditi@example.com |
|          4 | Neha Singh   |   23 | Female | neha@example.com  |
|          5 | Rohit Jain   |   26 | Male   | rohit@example.com |
+------------+--------------+------+--------+------------------+
```

Product Table

```
mysql> select*from Product;
+------------+----------------+---------------+
| product_id | product_name   | category      |
+------------+----------------+---------------+
|          1 | Smartphone X   | Electronics   |
|          2 | Organic Shampoo | Personal Care |
|          3 | Sports Shoes   | Footwear      |
+------------+----------------+---------------+
```

Survey Table

```
mysql> select*from Survey;
+------------+--------------+--------------+
| survey_id | survey_date | customer_id |
+------------+--------------+--------------+
|          1 | 2025-02-10  |            1 |
|          2 | 2025-02-12  |            2 |
|          3 | 2025-02-15  |            3 |
|          4 | 2025-02-20  |            4 |
+------------+--------------+--------------+
```


-

Feedback Table

```
mysql> select*from Feedback;
+-------------+-----------+------------+--------+--------------------+
| feedback_id | survey_id | product_id | rating | comments           |
+-------------+-----------+------------+--------+--------------------+
|           1 |         1 |          1 |      5 | Excellent product  |
|           2 |         2 |          2 |      4 | Good quality       |
|           3 |         3 |          3 |      3 | Average experience |
|           4 |         4 |          1 |      5 | Very satisfied     |
+-------------+-----------+------------+--------+--------------------+
```

2. Join Query

SELECT c.name, p.product_name, f.rating
FROM Customer c
JOIN Survey s ON c.customer_id = s.customer_id
JOIN Feedback f ON s.survey_id = f.survey_id
JOIN Product p ON f.product_id = p.product_id;

Result Outcome:

• The query executed successfully without any errors.
• Data was retrieved from the Customer, Survey, Feedback, and Product tables.
• The output displayed the following information:
o Customer Name
o Product Name
o Rating given by the customer
• JOIN operations correctly linked all tables using Customer ID, Survey ID, and Product ID.
• Only records with valid relationships across all tables were included in the result.
• The ratings shown corresponded accurately to the respective customer feedback for each product.
• The retrieved results matched the existing database records without inconsistencies.
• This confirms that the table relationships, foreign key constraints, and query logic are working correctly and the system is functioning as expected.

-

```
    -> JOIN Product p ON f.product_id = p.produ
+--------------+-----------------+--------+
| name         | product_name    | rating |
+--------------+-----------------+--------+
| Rahul Sharma | Smartphone X    |      5 |
| Sahil Verma  | Organic Shampoo |      4 |
| Aditi Mehta  | Sports Shoes    |      3 |
+--------------+-----------------+--------+
```

3. Create a View and Rollback

CREATE VIEW Product_Summary AS
SELECT
p.product_name,
AVG(f.rating) AS average_rating
FROM Product p
JOIN Feedback f ON p.product_id = f.product_id
GROUP BY p.product_name;
SELECT * FROM Product_Summary;

Result Outcome

• The view was created successfully without any errors.
• Data was retrieved from the Product and Feedback tables through the defined view.
• The output displayed the following information:
o Product Name
o Average Rating of each product
• The AVG() aggregation function correctly calculated the mean rating based on all feedback entries for each product.
• GROUP BY operation accurately grouped records according to product name.
• The view provided a summarized representation of product performance based on customer feedback.
• The results matched the underlying database records and reflected correct calculations.
• This confirms that aggregation functions, grouping logic, and view implementation are working correctly and efficiently.

-

```
mysql> SELECT * FROM Product_Summary;
+-----------------+----------------+
| product_name    | average_rating |
+-----------------+----------------+
| Smartphone X    |         5.0000 |
| Organic Shampoo |         4.0000 |
| Sports Shoes    |         3.0000 |
+-----------------+----------------+
```

Rollback:

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> INSERT INTO Feedback (survey_id, product_id, rating, comments)
    -> VALUES (1, 2, 10, 'Invalid rating test');
ERROR 3819 (HY000): Check constraint 'feedback_chk_1' is violated.
mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.03 sec)
```

The results of system testing confirmed that the database system operates efficiently and reliably:

• All SQL queries executed without errors when valid data was provided.

• Database constraints successfully maintained data integrity and consistency.

• Join queries and views returned accurate and meaningful results.

• Aggregation queries produced correct average ratings for products.

10.2 Constraint Validation

Database constraints were rigorously tested to ensure data accuracy, consistency, and integrity across all tables.

-

• **Primary Key Constraints**
Prevented duplicate entries in Customer, Product, Survey, and Feedback tables by enforcing unique record identification.

• **Foreign Key Constraints**
Ensured valid relationships between related tables, such as:
o Customer and Survey
o Survey and Feedback
o Product and Feedback

• **NOT NULL Constraints**
Ensured that mandatory fields such as:
o Customer name
o Product name
o Rating
were not left empty during data entry.

• **UNIQUE Constraints**
Where applied (e.g., email or specific identifiers), prevented duplicate values and maintained data uniqueness.

These constraints ensured that only valid, complete, and consistent data was stored in the database, thereby enhancing system reliability and preventing anomalies.

10.3 Sample Output Verification
Sample data was inserted into all tables to verify the correctness of outputs. Various SELECT queries were executed to retrieve customer, survey, product, and feedback information.

• Retrieved data was compared with the inserted records and found to be accurate.
• JOIN queries successfully combined data from multiple tables.
• The output clearly displayed:
o Customer names
o Product details
o Survey information
o Customer ratings and feedback

This confirmed that relationships among tables were properly established and data retrieval operations were functioning correctly.

-

10.4 Report Generation Result

Reports on product performance and customer feedback were generated using advanced SQL operations such as:

• JOIN queries for integrated data analysis
• GROUP BY and HAVING clauses for product-wise statistics
• Aggregate functions (e.g., AVG) for rating analysis
• Views for simplified, reusable, and efficient reporting

The generated reports accurately displayed:

• Product performance based on customer ratings
• Average rating for each product
• Customer feedback summaries

These reports helped in analyzing product quality, understanding customer satisfaction levels, and supporting informed decision-making.

Final Result

The system testing process confirmed that the Customer Feedback and Product Evaluation Database System operates efficiently and meets all project objectives. The system successfully:

• Maintains data integrity and consistency
• Ensures accurate data storage and processing
• Produces reliable and meaningful analytical results

Overall, the system proved to be robust, reliable, and effective for managing customer surveys, product feedback, and performance analysis.

-

11. SECURITY, BACKUP AND RECOVERY

Database security is implemented using role-based access control in MySQL. This ensures that users can access only the data and operations relevant to their responsibilities.

Key security measures include:

• Different user roles such as Administrator and Analyst/Staff are defined in the system.
• Each role is granted specific privileges using MySQL's GRANT command.
• Sensitive operations such as database modification, user management, and structure changes are restricted to administrators.
• Staff members are allowed to insert and view customer, survey, and feedback records but cannot alter critical system tables.
• Unauthorized access is prevented by revoking unnecessary privileges using the REVOKE command.
• Password authentication ensures that only authorized users can access the database.

Example of granting privileges:

```
GRANT SELECT, INSERT, UPDATE ON
MarketResearchDB.* TO 'staff_user'@'localhost';
```

Example of revoking privileges:

```
REVOKE DELETE ON MarketResearchDB.* FROM
'staff_user'@'localhost';
```

These controls help maintain data security, prevent misuse, and support accountability.

BACKUP

Database backup is essential to protect data against accidental deletion, hardware failure, or corruption.
The system uses **mysqldump**, a MySQL utility, to create complete backups of the database.

-

Key points of the backup process:

• Full database backups are taken periodically.
• Backup files contain both database structure and stored data.
• Backups are saved as SQL files, making them easy to store, share, and restore.
• Regular backups ensure data safety and business continuity.

Example backup command:`mysqldump -u root -p MarketResearchDB > MarketResearchDB_Backup.sql`

This command creates a backup file containing all tables and records of the database.

RECOVERY

Recovery refers to restoring the database from a backup file in case of data loss, corruption, or system failure.

Recovery features include:

• SQL backup files generated using mysqldump are used for restoration.
• The database can be recreated and populated using a single SQL file.
• Recovery ensures minimal downtime and prevents permanent data loss.
• The process is simple and reliable for database administrators.

Example recovery command:`mysql -u root -p MarketResearchDB < MarketResearchDB_Backup.sql`

This command restores the database to its previous state using the backup file.

SUMMARY

• Role-based access control ensures secure database usage.
• GRANT and REVOKE commands prevent unauthorized operations.
• mysqldump provides reliable database backups.
• SQL backup files allow easy and fast recovery.

-

• Together, these mechanisms ensure data security, reliability, and integrity of the Market Research Database Management System.

-

12. FUTURE SCOPE AND CONCLUSION

Future enhancements and concluding remarks are essential to evaluate the long-term usefulness and impact of the Market Research Database Management System. This section outlines potential improvements that can expand the system's capabilities as well as the overall conclusion drawn from the project implementation.

FUTURE SCOPE

The current system provides a solid database foundation for managing market research data; however, several enhancements can further improve usability, scalability, and analytical power.

Key future developments include:

• Development of a web-based interface to conduct online surveys and allow customers to submit responses remotely.
• Integration with a mobile application to enable real-time data collection through smartphones and tablets.
• Implementation of advanced analytics and data visualization tools to generate insights, trends, and graphical reports.
• Deployment on cloud platforms to support remote access, scalability, and improved data availability.
• Addition of automated reporting features for decision-makers and management teams.
• Integration with external systems such as CRM or business intelligence tools for comprehensive analysis.

These improvements would transform the system into a fully functional enterprise-level market research platform.

CONCLUSION

The Market Research Database Management System demonstrates the

-

effective use of MySQL for storing, managing, and analyzing customer survey and feedback data. The system ensures data accuracy, consistency, security, and efficient retrieval through well-designed tables, constraints, relationships, and queries.

By replacing manual record-keeping methods with an automated database solution, the project reduces errors, improves data organization, and supports faster decision-making. The implementation of transactions, views, and integrity constraints further enhances reliability and robustness.

Overall, the project provides a strong foundation for future expansion and real-world applications in market analysis, customer research, and product evaluation.

-

13. REFERENCES

Reliable sources were consulted during the development of this project to ensure correctness and adherence to database management best practices.

• MySQL Official Documentation — for SQL syntax, commands, and database features
• Database Management System textbooks — for theoretical concepts and design principles
• Online tutorials and learning resources — for practical examples and implementation guidance

These references helped in understanding database design, SQL operations, and real-world applications of relational database systems.

-

14: GLOSSARY

The glossary section provides simple and clear definitions of important technical terms used throughout the Market Research Database Management System project. These terms are mainly related to database systems, SQL concepts, and data management processes, which are essential for understanding how the system works. The definitions are written in easy and understandable language so that readers with little or no prior knowledge of database systems can easily comprehend them. This section serves as a quick reference guide and helps eliminate confusion regarding technical terminology used in the project documentation.

GLOSSARY

DBMS (Database Management System):
A Database Management System is software that allows users to store, organize, manage, and retrieve data efficiently. It provides tools for handling large volumes of data securely while maintaining data integrity and consistency.

SQL (Structured Query Language):
SQL is a standard programming language used to interact with relational databases. It is used to create database structures, insert and update data, retrieve records, and control access to the database.

Primary Key:
A primary key is a unique identifier for each record in a database table. It ensures that every record can be uniquely identified and prevents duplicate entries.

Foreign Key:
A foreign key is a field in one table that refers to the primary key of another table. It is used to establish relationships between tables and maintain referential integrity.

Normalization:
Normalization is the process of organizing data in a database to reduce redundancy and improve data consistency by dividing large tables into smaller related tables.

Transaction:
A transaction is a logical unit of work consisting of one or more database operations that are executed together. It ensures that all operations are completed successfully or none are applied, maintaining database consistency.

Concurrency:
Concurrency refers to the ability of a database system to allow multiple users to access and modify data simultaneously without causing conflicts or inconsistencies.

-